

09:00-09:50 (쉬는 시간 11:30-12:00)

Router는 파일 확장자가 아니라 명시된 업무 의도로 분기합니다

meeting_transcript와 code_diff를 같은 entry point에서 실행한다.

진행

강의 12분 · 강사 시연 10분 · 소프트웨어
실습 23분 · 실행 확인 5분

사용 파일

src/course_services/service_router.py
src/course_services/meeting_service.py

확인할 결과

unified_service_result.json

5일차는 여덟 개 차시로 하나의 서비스를 완성합니다

오전 3개 차시 뒤 30분 휴식, 12:00-13:00 점심, 오후 5개 차시 뒤 휴식과 Q&A로 마무리합니다.

시간	차시	배우는 내용	진행	확인할 결과
09:00-09:50	1차시	Router는 파일 확장자가 아니라 명시된 업무 의도로 분기합니다	강의 12 시연 10 소프트웨어 실습 23 확인 5분	unified_service_result.json
09:50-10:40	2차시	LangSmith trace는 실행 과정을 질문할 수 있게 만듭니다	강의 12 시연 10 소프트웨어 실습 23 확인 5분	trace.json 또는 LangSmith run
10:40-11:30	3차시	운영 dashboard는 예쁜 그래프보다 결정 가능한 신호가 먼저입니다	강의 12 시연 10 소프트웨어 실습 23 확인 5분	monitoring_view.md
13:00-13:50	4차시	평가는 같은 dataset으로 baseline과 candidate를 비교해야 합니다	강의 12 시연 10 소프트웨어 실습 23 확인 5분	experiment_result.json
13:50-14:40	5차시	사람 피드백은 좋아요 버튼보다 판단 이유가 필요합니다	강의 12 시연 10 소프트웨어 실습 23 확인 5분	human_feedback.jsonl
14:40-15:30	6차시	운영 가능한 Agent는 개인정보와 장애 대응을 먼저 설명합니다	강의 12 시연 10 소프트웨어 실습 23 확인 5분	ops_checklist.md
15:30-16:20	7차시	배포 후보는 코드·문서·demo가 같은 버전을 가리켜야 합니다	강의 12 시연 10 소프트웨어 실습 23 확인 5분	release_candidate
16:20-17:10	8차시	최종 발표는 모델 자랑보다 문제·증거·운영 결정을 보여줍니다	강의 12 시연 10 소프트웨어 실습 23 확인 5분	final_tag_and_scorecard

11:30-12:00 쉬는 시간 · 12:00-13:00 점심시간 · 17:10-17:40 쉬는 시간 · 17:40-18:00 Q&A·실행 복구

모델 추측으로 업무 종류를 고르면 잘못된 tool과 권한이 연결될 수 있다.

meeting_transcript와 code_diff를 같은 entry point에서
실행한다.

이번 차시의 확인 결과: unified_service_result.json

이 차시가 끝나면 세 가지를 설명하고 실행할 수 있습니다

- 01 — meeting_transcript와 code_diff를 같은 entry point에서 실행한다.
- 02 — 내용을 LLM에게 보여주고 업무 종류와 권한을 함께 추측시킨다. 왜 위험한지 설명한다.
- 03 — unified_service_result.json에서 정상과 실패 상태를 직접 확인한다.

처음 보는 용어는 이 네 가지 역할만 구분합니다

용어	이 수업에서 쓰는 뜻
Router	입력을 알맞은 workflow로 전달
Input kind	호출자가 명시한 업무 종류
Adapter	서로 다른 service 결과를 연결
Boundary	읽기·쓰기·승인 경계

입력·처리·결과·검증을 분리하면 문제가 빨리 보입니다

INPUT

src/course_services/service_router.py

PROCESS

path validate → input kind 확인 → meeting 또는 review

OUTPUT

unified_service_result.json

VERIFY

회의와 diff가 각각 올바른 service 이름으로 반환된다.
unknown kind와 workspace escape는
EXPECTED_FAILURE다.

실행 흐름은 다섯 단계로 고정합니다

01

path validate

02

input kind 확인

03

meeting 또는
review

04

공통 result

05

사람 승인

마지막 판단은 unified_service_result.json에서 사람이 확인합니다.

코드보다 먼저 성공·실패 계약을 정합니다

- 정상 — 회의와 diff가 각각 올바른 service 이름으로 반환된다.
- 경계 — unknown kind와 workspace escape는 EXPECTED_FAILURE다.
- 외부 쓰기 — 기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출
- 기록 — status·error_code·입력 식별자·unified_service_result.json

어떤 파일을 열고 어디에서 결과를 확인하는지 먼저 봅시다

입력·fixture

```
src/course_services/service_router.py
```

구현 코드

```
src/course_services/meeting_service.py
```

검증·test

```
src/course_services/review_service.py
```

따라하기 notebook

```
materials/day5/day5_service_lab.ipynb
```

Router는 파일 확장자가 아니라 명시된 업무 의도로 분기합니다

모델 추측으로 업무 종류를 고르면 잘못된 tool과 권한이 연결될 수 있다.

input_kind를 API contract로 받고 unsupported type은 거부한다.

이 차시에서
버릴 오해

내용을 LLM에게
보여주고 업무 종류와
권한을 함께 추측시킨
다.

비슷해 보이는 선택도 책임과 검증 범위가 다릅니다

구분	역할	이번 차시의 판단 기준
Router	입력을 알맞은 workflow로 전달	결정론 test로 먼저 확인
Input kind	호출자가 명시한 업무 종류	adapter 경계를 확인
Adapter	서로 다른 service 결과를 연결	비용·지연·권한을 확인
Boundary	읽기·쓰기·승인 경계	사람 판단과 운영 기록을 확인

가장 먼저 재현할 실패는 이것입니다

실패

내용을 LLM에게 보여주고 업무
종류와 권한을 함께 추측시킨다.

사용자 영향

모델 추측으로 업무 종류를 고르면 잘못된
tool과 권한이 연결될 수 있다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

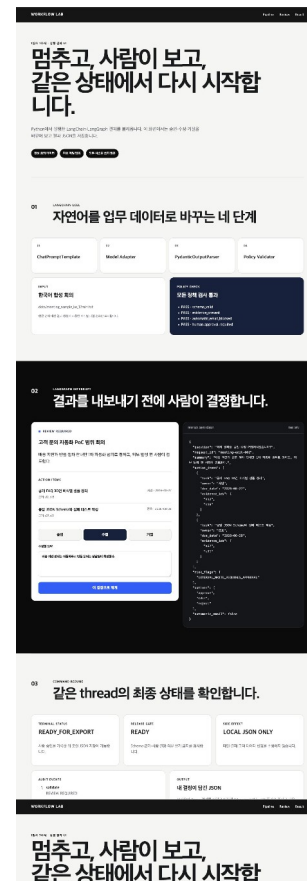
복구는 재시도보다 먼저 중단 조건을 정합니다

- 01 — 탐지: unknown kind와 workspace escape는 EXPECTED_FAILURE다.
- 02 — 중단: 내용을 LLM에게 보여주고 업무 종류와 권한을 함께 추측시킨다.
- 03 — 복구: input_kind를 API contract로 받고 unsupported type은 거부한다.
- 04 — 증거: unified_service_result.json과 test 결과를 함께 남긴다.

Codex는 코드를 대신 쓰는 도구보다 검증 루프에 가깝습니다

새 service route를 추가할 때
workspace 경계와
external_write=false를 test한다.

저장소 이해
→ 허용 범위 변경
→ focused test
→ diff 리뷰
→ 사람 merge



Codex에게는 목표보다 완료 조건을 더 구체적으로 줍니다

목표

새 service route를 추가할 때 workspace 경계와 external_write=false를 test한다.

허용 경로

src/course_services/meeting_service.py
src/course_services/review_service.py

완료 조건

회의와 diff가 각각 올바른 service 이름으로 반환된다.
unknown kind와 workspace escape는 EXPECTED_FAILURE다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

생성된 patch는 diff와 test 증거를 보고 사람이 결정합니다

- 01 **Scope** — 허용 경로 밖 파일이 바뀌지 않았는가
- 02 **Focused test** — 회의와 diff가 각각 올바른 service 이름으로 반환된다.
- 03 **Boundary test** — unknown kind와 workspace escape는 EXPECTED_FAILURE다.
- 04 **Human merge** — Codex 리뷰와 test 통과는 자동 승인이 아니다

강사가 먼저 정상 경로를 끝까지 실행합니다

- 01 — 열기: `src/course_services/service_router.py`
- 02 — 구현 확인: `src/course_services/meeting_service.py`
- 03 — 실행: `python -m pytest -q tests/test_course_services.py -k router`
- 04 — 성공 신호: 회의와 diff가 각각 올바른 service 이름으로 반환된다.

같은 저장소에서 이 명령을 실행합니다

```
$ python -m pytest -q tests/test_course_services.py -k router
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지

완료 여부는 화면이 아니라 결과 계약으로 확인합니다

STATUS

SUCCESS 또는 EXPECTED_FAILURE

ARTIFACT

unified_service_result.json

사람이 확인할 세 줄

1. 회의와 diff가 각각 올바른 service 이름으로 반환된다.
2. unknown kind와 workspace escape는 EXPECTED_FAILURE다.
3. external_write=false

강사는 실패 한 건도 바로 재현하고 복구합니다

재현

내용을 LLM에게 보여주고 업무 종류와 권한을 함께 추측시킨다.

복구

input_kind를 API contract로 받고 unsupported type은 거부한다.

확인: unknown kind와 workspace escape는 EXPECTED_FAILURE다.

이제 같은 코드를 각자 실행하고 한 줄만 바꿉니다

열 파일

materials/day5/day5_service_lab.ipynb

수정 범위

notebook의 실습 변수 또는 fixture 한 줄 · src 전체를 복사해 바꾸지 않음

완료 기준

회의와 diff가 각각 올바른 service 이름으로 반환된다.
unified_service_result.json 저장

STEP 1 · 입력과 설정을 확인합니다

파일: `src/course_services/service_router.py`
변수: 실행 경로·`provider·dry-run` 여부

결과 파일: `unified_service_result.json`

STEP 2 · 정상 경로를 실행합니다

```
python -m pytest -q tests/test_course_services.py -k router
```

성공 신호: 회의와 diff가 각각 올바른 service 이름으로 반환된다.

결과 파일: unified_service_result.json

STEP 3 · 경계값을 바꿔 실패를 확인합니다

unknown kind와 workspace escape는 EXPECTED_FAILURE다.
복구: input_kind를 API contract로 받고 unsupported type은 거부한다.

결과에 error_code와 external_write=false가 보이면 완료

정상 case를 focused test로 확인합니다

NORMAL

회의와 diff가 각각 올바른 service 이름으로 반환된다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

가장 중요한 실패 조건을 test로 고정합니다

BOUNDARY

unknown kind와 workspace escape는 EXPECTED_FAILURE다.

- 01 — raw traceback 대신 stable error_code가 남는가
- 02 — 외부 쓰기·자동 메일이 false인가
- 03 — 실패가 다음 차시에서 재현 가능한가

재직자는 작은 운영 문제 한 곳에 먼저 적용합니다

한 화면에서 회의·PR 리뷰를 선택 실행하는 내부 도구

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — unified_service_result.json을 운영 검토 자료로 사용

구직자는 제품 판단과 검증 증거를 함께 보여줍니다

한 화면에서 회의·PR 리뷰를 선택 실행하는 내부 도구

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — unified_service_result.json과 README 재현 절차를 제출

서비스로 운영하려면 이 네 가지 질문에 답해야 합니다

품질

무엇을 회의와 diff가 각각 올바른 service 이름으로 반환된다.로 정의하는가

안전

어디에서 사람 승인과 external_write=false를 확인하는가

복구

input_kind를 API contract로 받고 unsupported type은 거부한다.

관측

unified_service_result.json에서 어떤 status:error_code를 보는가

1차시를 마치기 전에 세 가지를 확인합니다

- 01 — 설명: Router는 파일 확장자가 아니라 명시된 업무 의도로 분기합니다이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_course_services.py -k router`
- 03 — 증거: `unified_service_result.json`과 정상·실패 test 결과가 남아 있다.

다음 차시: LangSmith trace는 실행 과정을 질문할 수 있게 만듭니다

LangSmith trace는 실행 과정을 질문할 수 있게 만듭니다

project·trace·run·thread의 관계를 이해하고 첫 trace를 남긴다.

진행

강의 12분 · 강사 시연 10분 · 소프트웨어
실습 23분 · 실행 확인 5분

사용 파일

src/observability_lab.py
src/workflow_service.py

확인할 결과

trace.json 또는 LangSmith run

2차시는 앞의 결과를 이어받아 다음 상태를 만듭니다

입력	——	unified_service_result.json
이번 차시	——	LangSmith trace는 실행 과정을 질문할 수 있게 만듭니다
결과	——	trace.json 또는 LangSmith run
다음 연결	——	운영 dashboard는 예쁜 그래프보다 결정 가능한 신호가 먼저입니다

최종 JSON만 저장하면 느린 node와 실패 원인을 다시 찾기 어렵다.

project·trace·run·thread의 관계를 이해하고 첫 trace를 남긴다.

이번 차시의 확인 결과: trace.json 또는 LangSmith run

이 차시가 끝나면 세 가지를 설명하고 실행할 수 있습니다

- 01 — `project·trace·run·thread`의 관계를 이해하고 첫 `trace`를 남긴다.
- 02 — `raw transcript·token·고객` 정보를 `trace input`에 그대로 보낸다. 왜 위험한지 설명한다.
- 03 — `trace.json` 또는 `LangSmith run`에서 정상과 실패 상태를 직접 확인한다.

처음 보는 용어는 이 네 가지 역할만 구분합니다

용어	이 수업에서 쓰는 뜻
Project	관련 실행을 모으는 공간
Trace	한 요청의 전체 실행 계보
Run	node·LLM·tool 한 번의 실행
Thread	대화·재개 실행의 연속성

입력·처리·결과·검증을 분리하면 문제가 빨리 보입니다

INPUT

src/observability_lab.py

OUTPUT

trace.json 또는 LangSmith run

PROCESS

request start → span 시작 → 입력 최소화

VERIFY

trace에 span 계층과 READY/HOLD 결정이 남는다.
local_only나 key 없음은 upload하지 않고 error code를 반환한다.

실행 흐름은 다섯 단계로 고정합니다

01

request start

02

span 시작

03

입력 최소화

04

status·latency

05

trace close

마지막 판단은 trace.json 또는 LangSmith run에서 사람이 확인합니다.

코드보다 먼저 성공·실패 계약을 정합니다

- 정상 ——— trace에 span 계층과 READY/HOLD 결정이 남는다.
- 경계 ——— local_only나 key 없음은 upload하지 않고 error code를 반환한다.
- 외부 쓰기 ——— 기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출
- 기록 ——— status·error_code·입력 식별자·trace.json 또는 LangSmith run

어떤 파일을 열고 어디에서 결과를 확인하는지 먼저 봅시다

입력·fixture

```
src/observability_lab.py
```

구현 코드

```
src/workflow_service.py
```

검증·test

```
output/day1-workflow/trace.json
```

따라하기 notebook

```
.env.sample
```

LangSmith trace는 실행 과정을 질문할 수 있게 만듭니다

최종 JSON만 저장하면 느린 node와 실패 원인을 다시 찾기 어렵다.

local trace 기본, synthetic/deidentified만 명시 upload, raw content는 제외한다.

이 차시에서
버릴 오해

raw transcript·token·
고객 정보를 trace input
에 그대로 보낸다.

비슷해 보이는 선택도 책임과 검증 범위가 다릅니다

구분	역할	이번 차시의 판단 기준
Project	관련 실행을 모으는 공간	결정론 test로 먼저 확인
Trace	한 요청의 전체 실행 계보	adapter 경계를 확인
Run	node·LLM·tool 한 번의 실행	비용·지연·권한을 확인
Thread	대화·재개 실행의 연속성	사람 판단과 운영 기록을 확인

가장 먼저 재현할 실패는 이것입니다

실패

raw transcript-token·고객 정보를
trace input에 그대로 보낸다.

사용자 영향

최종 JSON만 저장하면 느린 node와 실패
원인을 다시 찾기 어렵다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

복구는 재시도보다 먼저 중단 조건을 정합니다

- 01 — 탐지: local_only나 key 없음은 upload하지 않고 error code를 반환한다.
- 02 — 중단: raw transcript·token·고객 정보를 trace input에 그대로 보낸다.
- 03 — 복구: local trace 기본, synthetic/deidentified만 명시 upload, raw content는 제외한다.
- 04 — 증거: trace.json 또는 LangSmith run과 test 결과를 함께 남긴다.

Codex는 코드를 대신 쓰는 도구보다 검증 루프에 가깝습니다

새 node에 latency-status metadata를 추가하되 PII payload를 넣지 않는다.

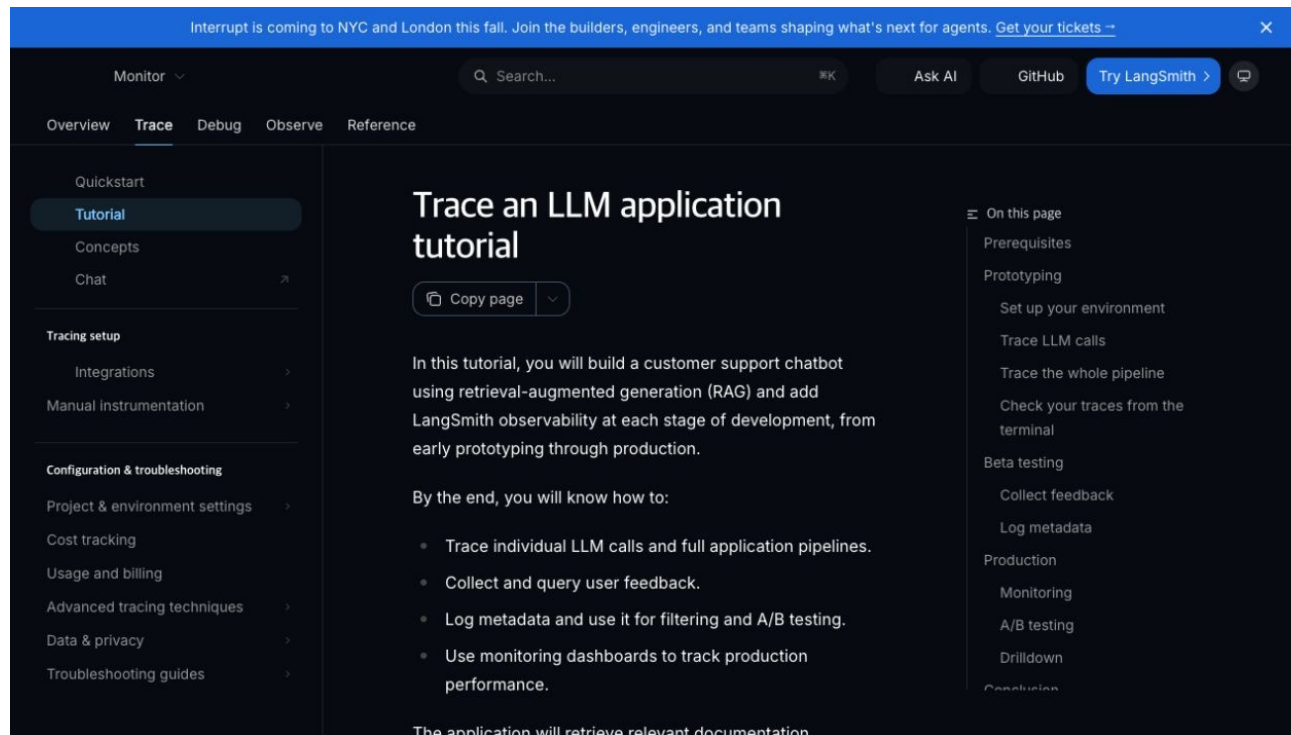
저장소 이해

→ 허용 범위 변경

→ focused test

→ diff 리뷰

→ 사람 merge



Codex에게는 목표보다 완료 조건을 더 구체적으로 줍니다

목표

새 node에 latency-status metadata를 추가하되 PII payload를 넣지 않는다.

허용 경로

src/workflow_service.py
output/day1-workflow/trace.json

완료 조건

trace에 span 계층과 READY/HOLD 결정이 남는다.
local_only나 key 없음은 upload하지 않고 error code를 반환한다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

생성된 patch는 diff와 test 증거를 보고 사람이 결정합니다

- 01 **Scope** — 허용 경로 밖 파일이 바뀌지 않았는가
- 02 **Focused test** — trace에 span 계층과 READY/HOLD 결정이 남는다.
- 03 **Boundary test** — local_only나 key 없음은 upload하지 않고 error code를 반환한다.
- 04 **Human merge** — Codex 리뷰와 test 통과는 자동 승인이 아니다

강사가 먼저 정상 경로를 끝까지 실행합니다

- 01 — 열기: `src/observability_lab.py`
- 02 — 구현 확인: `src/workflow_service.py`
- 03 — 실행: `python -m src.workflow_service --decision approve --out output/day5-trace`
- 04 — 성공 신호: trace에 span 계층과 READY/HOLD 결정이 남는다.

같은 저장소에서 이 명령을 실행합니다

```
$ python -m src.workflow_service --decision approve --out output/day5-trace
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지

완료 여부는 화면이 아니라 결과 계약으로 확인합니다

STATUS

SUCCESS 또는 EXPECTED_FAILURE

ARTIFACT

trace.json 또는 LangSmith run

사람이 확인할 세 줄

1. trace에 span 계층과 READY/HOLD 결정이 남는다.
2. local_only나 key 없음은 upload하지 않고 error code를 반환한다.
3. external_write=false

강사는 실패 한 건도 바로 재현하고 복구합니다

재현

raw transcript·token·고객 정보를 trace input에 그대로 보낸다.

복구

local trace 기본,
synthetic/deidentified만 명시 upload,
raw content는 제외한다.

확인: local_only나 key 없음은 upload하지 않고 error code를 반환한다.

이제 같은 코드를 각자 실행하고 한 줄만 바꿉니다

열 파일 `.env.sample`

수정 범위 notebook의 실습 변수 또는 fixture 한 줄 · src 전체를 복사해 바꾸지 않음

완료 기준 trace에 span 계층과 READY/HOLD 결정이 남는다.
trace.json 또는 LangSmith run 저장

STEP 1 · 입력과 설정을 확인합니다

파일: `src/observability_lab.py`
변수: 실행 경로·provider·dry-run 여부

결과 파일: `trace.json` 또는 `LangSmith run`

STEP 2 · 정상 경로를 실행합니다

```
python -m src.workflow_service --decision approve --out output/day5-trace
```

성공 신호: trace에 span 계층과 READY/HOLD 결정이 남는다.

결과 파일: trace.json 또는 LangSmith run

STEP 3 · 경계값을 바꿔 실패를 확인합니다

local_only나 key 없음은 upload하지 않고 error code를 반환한다.
복구: local trace 기본, synthetic/deidentified만 명시 upload, raw content는 제외한다.

결과에 error_code와 external_write=false가 보이면 완료

정상 case를 focused test로 확인합니다

NORMAL

trace에 span 계층과 READY/HOLD 결정이 남는다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

가장 중요한 실패 조건을 test로 고정합니다

BOUNDARY

local_only나 key 없음은 upload하지 않고 error code를 반환한다.

- 01 — raw traceback 대신 stable error_code가 남는가
- 02 — 외부 쓰기·자동 메일이 false인가
- 03 — 실패가 다음 차시에서 재현 가능한가

재직자는 작은 운영 문제 한 곳에 먼저 적용합니다

장애 문의에서 request ID로 어느 node가 실패했는지 조회

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — trace.json 또는 LangSmith run을 운영 검토 자료로 사용

구직자는 제품 판단과 검증 증거를 함께 보여줍니다

장애 문의에서 request ID로 어느 node가 실패했는지 조회

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — trace.json 또는 LangSmith run과 README 재현 절차를 제출

서비스로 운영하려면 이 네 가지 질문에 답해야 합니다

품질

무엇을 trace에 span 계층과 READY/HOLD 결정이 남는다.로 정의하는가

안전

어디에서 사람 승인과 external_write=false를 확인하는가

복구

local trace 기본, synthetic/deidentified만 명시 upload, raw content는 제외한다.

관측

trace.json 또는 LangSmith run에서 어떤 status:error_code를 보는가

2차시를 마치기 전에 세 가지를 확인합니다

- 01 — 설명: LangSmith trace는 실행 과정을 질문할 수 있게 만듭니다이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m src.workflow_service --decision approve --out output/day5-trace`
- 03 — 증거: `trace.json` 또는 LangSmith run과 정상·실패 test 결과가 남아 있다.

다음 차시: 운영 dashboard는 예쁜 그래프보다 결정 가능한 신호가 먼저입니다

10:40-11:30 (쉬는 시간 11:30-12:00)

운영 dashboard는 예쁜 그래프보다 결정 가능한 신호가 먼저입니다

status·error·latency·provider·cost metadata를 비교한다.

진행

강의 12분 · 강사 시연 10분 · 소프트웨어
실습 23분 · 실행 확인 5분

사용 파일

src/observability_lab.py
output/day1-workflow/trace.json

확인할 결과

monitoring_view.md

3차시는 앞의 결과를 이어받아 다음 상태를 만듭니다

입력	————	trace.json 또는 LangSmith run
이번 차시	————	운영 dashboard는 예쁜 그래프보다 결정 가능한 신호가 먼저입니다
결과	————	monitoring_view.md
다음 연결	————	평가는 같은 dataset으로 baseline과 candidate를 비교해야 합니다

평균 latency 하나만 보면 특정 provider와 실패 유형의 꼬리가 숨겨진다.

status·error·latency·provider·cost metadata를 비교한다.

이번 차시의 확인 결과: monitoring_view.md

이 차시가 끝나면 세 가지를 설명하고 실행할 수 있습니다

- 01 — status·error·latency·provider·cost metadata를 비교한다.
- 02 — 성공 trace만 보고 HOLD·reject·fallback을 제외한다. 왜 위험한지 설명한다.
- 03 — monitoring_view.md에서 정상과 실패 상태를 직접 확인한다.

처음 보는 용어는 이 네 가지 역할만 구분합니다

용어	이 수업에서 쓰는 뜻
Tag	실행을 분류하는 짧은 label
Metadata	조회·비교용 구조화 값
Latency	요청부터 결과까지 걸린 시간
Cost	token·compute 사용 비용

입력·처리·결과·검증을 분리하면 문제가 빨리 보입니다

INPUT

src/observability_lab.py

PROCESS

tag version → provider metadata → node latency

OUTPUT

monitoring_view.md

VERIFY

provider_used와 node latency를 필터링할 수 있다.
오류 원문 대신 stable error_code와 type만 외부 trace
에 남는다.

실행 흐름은 다섯 단계로 고정합니다

01

tag version

02

provider metadata

03

node latency

04

error code

05

READY/HOLD filter

마지막 판단은 `monitoring_view.md`에서 사람이 확인합니다.

코드보다 먼저 성공·실패 계약을 정합니다

- 정상 ——— provider_used와 node latency를 필터링할 수 있다.
- 경계 ——— 오류 원문 대신 stable error_code와 type만 외부 trace에 남는다.
- 외부 쓰기 ——— 기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출
- 기록 ——— status·error_code·입력 식별자·monitoring_view.md

어떤 파일을 열고 어디에서 결과를 확인하는지 먼저 봅시다

입력·fixture

src/observability_lab.py

구현 코드

output/day1-workflow/trace.json

검증·test

materials/day5/day5_service_lab.ipynb

따라하기 notebook

AGENTS.md

운영 dashboard는 예쁜 그래프보다 결정 가능한 신호가 먼저입니다

평균 latency 하나만 보면 특정 provider와 실패 유형의 꼬리가 숨겨진다.

status와 fallback_reason을 같은 dashboard filter에 포함한다.

이 차시에서
버릴 오해

성공 trace만 보고
HOLD·reject·fallback을
제외한다.

비슷해 보이는 선택도 책임과 검증 범위가 다릅니다

구분	역할	이번 차시의 판단 기준
Tag	실행을 분류하는 짧은 label	결정론 test로 먼저 확인
Metadata	조회·비교용 구조화 값	adapter 경계를 확인
Latency	요청부터 결과까지 걸린 시간	비용·지연·권한을 확인
Cost	token·compute 사용 비용	사람 판단과 운영 기록을 확인

가장 먼저 재현할 실패는 이것입니다

실패

성공 trace만 보고
HOLD·reject·fallback을 제외한다.

사용자 영향

평균 latency 하나만 보면 특정 provider와
실패 유형의 꼬리가 숨겨진다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

복구는 재시도보다 먼저 중단 조건을 정합니다

- 01 — 탐지: 오류 원문 대신 stable error_code와 type만 외부 trace에 남는다.
- 02 — 중단: 성공 trace만 보고 HOLD·reject·fallback을 제외한다.
- 03 — 복구: status와 fallback_reason을 같은 dashboard filter에 포함한다.
- 04 — 증거: monitoring_view.md과 test 결과를 함께 남긴다.

Codex는 코드를 대신 쓰는 도구보다 검증 루프에 가깝습니다

추적 field를 늘리기 전에 운영
질문 한 개와 필요한 metadata만
정의한다.

저장소 이해

→ 허용 범위 변경

→ focused test

→ diff 리뷰

→ 사람 merge

Codex에게는 목표보다 완료 조건을 더 구체적으로 줍니다

목표

추적 field를 늘리기 전에 운영 질문 한 개와 필요한 metadata만 정의한다.

허용 경로

output/day1-workflow/trace.json
materials/day5/day5_service_lab.ipynb

완료 조건

provider_used와 node latency를 필터링할 수 있다.
오류 원문 대신 stable error_code와 type만 외부 trace에 남는다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

생성된 patch는 diff와 test 증거를 보고 사람이 결정합니다

- 01 **Scope** — 허용 경로 밖 파일이 바뀌지 않았는가
- 02 **Focused test** — provider_used와 node latency를 필터링할 수 있다.
- 03 **Boundary test** — 오류 원문 대신 stable error_code와 type만 외부 trace에 남는다.
- 04 **Human merge** — Codex 리뷰와 test 통과는 자동 승인이 아니다

강사가 먼저 정상 경로를 끝까지 실행합니다

- 01 — 열기: `src/observability_lab.py`
- 02 — 구현 확인: `output/day1-workflow/trace.json`
- 03 — 실행: `python -m pytest -q tests/test_observability_workflow.py`
- 04 — 성공 신호: `provider_used`와 `node latency`를 필터링할 수 있다.

같은 저장소에서 이 명령을 실행합니다

```
$ python -m pytest -q tests/test_observability_workflow.py
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지

완료 여부는 화면이 아니라 결과 계약으로 확인합니다

STATUS

SUCCESS 또는 EXPECTED_FAILURE

ARTIFACT

monitoring_view.md

사람이 확인할 세 줄

1. provider_used와 node latency를 필터링할 수 있다.
2. 오류 원문 대신 stable error_code와 type만 외부 trace에 남는다.
3. external_write=false

강사는 실패 한 건도 바로 재현하고 복구합니다

재현

성공 trace만 보고 HOLD·reject·fallback
을 제외한다.

복구

status와 fallback_reason을 같은
dashboard filter에 포함한다.

확인: 오류 원문 대신 stable error_code와 type만 외부 trace에 남는다.

이제 같은 코드를 각자 실행하고 한 줄만 바꿉니다

열 파일 AGENTS.md

수정 범위 notebook의 실습 변수 또는 fixture 한 줄 · src 전체를 복사해 바꾸지 않음

완료 기준 provider_used와 node latency를 필터링할 수 있다.
monitoring_view.md 저장

STEP 1 · 입력과 설정을 확인합니다

파일: `src/observability_lab.py`
변수: 실행 경로·provider·dry-run 여부

결과 파일: `monitoring_view.md`

STEP 2 · 정상 경로를 실행합니다

```
python -m pytest -q tests/test_observability_workflow.py
```

성공 신호: provider_used와 node latency를 필터링할 수 있다.

결과 파일: monitoring_view.md

STEP 3 · 경계값을 바꿔 실패를 확인합니다

오류 원문 대신 `stable error_code`와 `type`만 외부 trace에 남는다.
복구: `status`와 `fallback_reason`을 같은 dashboard filter에 포함한다.

결과에 `error_code`와 `external_write=false`가 보이면 완료

정상 case를 focused test로 확인합니다

NORMAL

provider_used와 node latency를 필터링할 수 있다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

가장 중요한 실패 조건을 test로 고정합니다

BOUNDARY

오류 원문 대신 `stable error_code`와 `type`만 외부 `trace`에 남는다.

- 01 — raw traceback 대신 `stable error_code`가 남는가
- 02 — 외부 쓰기·자동 메일이 `false`인가
- 03 — 실패가 다음 차시에서 재현 가능한가

재직자는 작은 운영 문제 한 곳에 먼저 적용합니다

모델 교체 후 latency-fallback-HOLD 비율 비교

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — monitoring_view.md을 운영 검토 자료로 사용

구직자는 제품 판단과 검증 증거를 함께 보여줍니다

모델 교체 후 latency-fallback-HOLD 비율 비교

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — monitoring_view.md과 README 재현 절차를 제출

서비스로 운영하려면 이 네 가지 질문에 답해야 합니다

품질

무엇을 `provider_used`와 `node latency`를 필터링할 수 있다.로 정의하는가

복구

`status`와 `fallback_reason`을 같은 `dashboard filter`에 포함한다.

안전

어디에서 사람 승인과 `external_write=false`를 확인하는가

관측

`monitoring_view.md`에서 어떤 `status.error_code`를 보는가

3차시를 마치기 전에 세 가지를 확인합니다

- 01 — 설명: 운영 dashboard는 예쁜 그래프보다 결정 가능한 신호가 먼저입니다! 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_observability_workflow.py`
- 03 — 증거: `monitoring_view.md`과 정상·실패 test 결과가 남아 있다.

다음 차시: 평가는 같은 dataset으로 baseline과 candidate를 비교해야 합니다

평가는 같은 dataset으로 baseline과 candidate를 비교해야 합니다

golden review case를 code evaluator로 실행한다.

진행

강의 12분 · 강사 시연 10분 · 소프트웨어
실습 23분 · 실행 확인 5분

사용 파일

src/course_services/eval_service.py
data/day5_eval/
golden_review_findings.json

확인할 결과

experiment_result.json

4차시는 앞의 결과를 이어받아 다음 상태를 만듭니다

입력	———	monitoring_view.md
이번 차시	———	평가는 같은 dataset으로 baseline과 candidate를 비교해야 합니다
결과	———	experiment_result.json
다음 연결	———	사람 피드백은 좋아요 버튼보다 판단 이유가 필요합니다

데모 입력이 매번 달라지면 개선인지 운인지 구분할 수 없다.

golden review case를 code evaluator로 실행한다.

이번 차시의 확인 결과: experiment_result.json

이 차시가 끝나면 세 가지를 설명하고 실행할 수 있습니다

- 01 — golden review case를 code evaluator로 실행한다.
- 02 — LLM judge 한 개 점수를 객관적 정답처럼 사용한다. 왜 위험한지 설명한다.
- 03 — experiment_result.json에서 정상과 실패 상태를 직접 확인한다.

처음 보는 용어는 이 네 가지 역할만 구분합니다

용어	이 수업에서 쓰는 뜻
Dataset	반복 평가할 입력·정답 묶음
Experiment	특정 버전을 dataset에 실행
Code evaluator	규칙으로 자동 채점
Candidate	새로 비교할 구현

입력·처리·결과·검증을 분리하면 문제가 빨리 보입니다

INPUT

src/course_services/eval_service.py

OUTPUT

experiment_result.json

PROCESS

golden load → baseline run → candidate run

VERIFY

fixture precision·recall·F1이 1.0이다.
precision 또는 recall 0.8 미만은 HOLD다.

실행 흐름은 다섯 단계로 고정합니다

01

golden load

02

baseline run

03

candidate run

04

metric compare

05

release gate

마지막 판단은 experiment_result.json에서 사람이 확인합니다.

코드보다 먼저 성공·실패 계약을 정합니다

정상	—— fixture precision·recall·F1이 1.0이다.
경계	—— precision 또는 recall 0.8 미만은 HOLD다.
외부 쓰기	—— 기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출
기록	—— status·error_code·입력 식별자·experiment_result.json

어떤 파일을 열고 어디에서 결과를 확인하는지 먼저 봅시다

입력·fixture

```
src/course_services/eval_service.py
```

구현 코드

```
data/day5_eval/golden_review_findings.json
```

검증·test

```
tests/test_course_services.py
```

따라하기 notebook

```
materials/day5/day5_service_lab.ipynb
```

평가는 같은 dataset으로 baseline과 candidate를 비교해야 합니다

데모 입력이 매번 달라지면 개선인지 운인지 구분할 수 없다.

schema-line mapping-policy는 code evaluator, 의미 품질은 human sample로 나눈다.

이 차시에서
버릴 오해

LLM judge 한 개 점수를
객관적 정답처럼
사용한다.

비슷해 보이는 선택도 책임과 검증 범위가 다릅니다

구분	역할	이번 차시의 판단 기준
Dataset	반복 평가할 입력·정답 묶음	결정론 test로 먼저 확인
Experiment	특정 버전을 dataset에 실행	adapter 경계를 확인
Code evaluator	규칙으로 자동 채점	비용·지연·권한을 확인
Candidate	새로 비교할 구현	사람 판단과 운영 기록을 확인

가장 먼저 재현할 실패는 이것입니다

실패

LLM judge 한 개 점수를 객관적
정답처럼 사용한다.

사용자 영향

데모 입력이 매번 달라지면 개선인지 운인지
구분할 수 없다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

복구는 재시도보다 먼저 중단 조건을 정합니다

- 01 — 탐지: precision 또는 recall 0.8 미만은 HOLD다.
- 02 — 중단: LLM judge 한 개 점수를 객관적 정답처럼 사용한다.
- 03 — 복구: schema·line mapping·policy는 code evaluator, 의미 품질은 human sample로 나눈다.
- 04 — 증거: experiment_result.json과 test 결과를 함께 남긴다.

Codex는 코드를 대신 쓰는 도구보다 검증 루프에 가깝습니다

새 평가 지표에 empty set과 threshold boundary test를 추가한다.

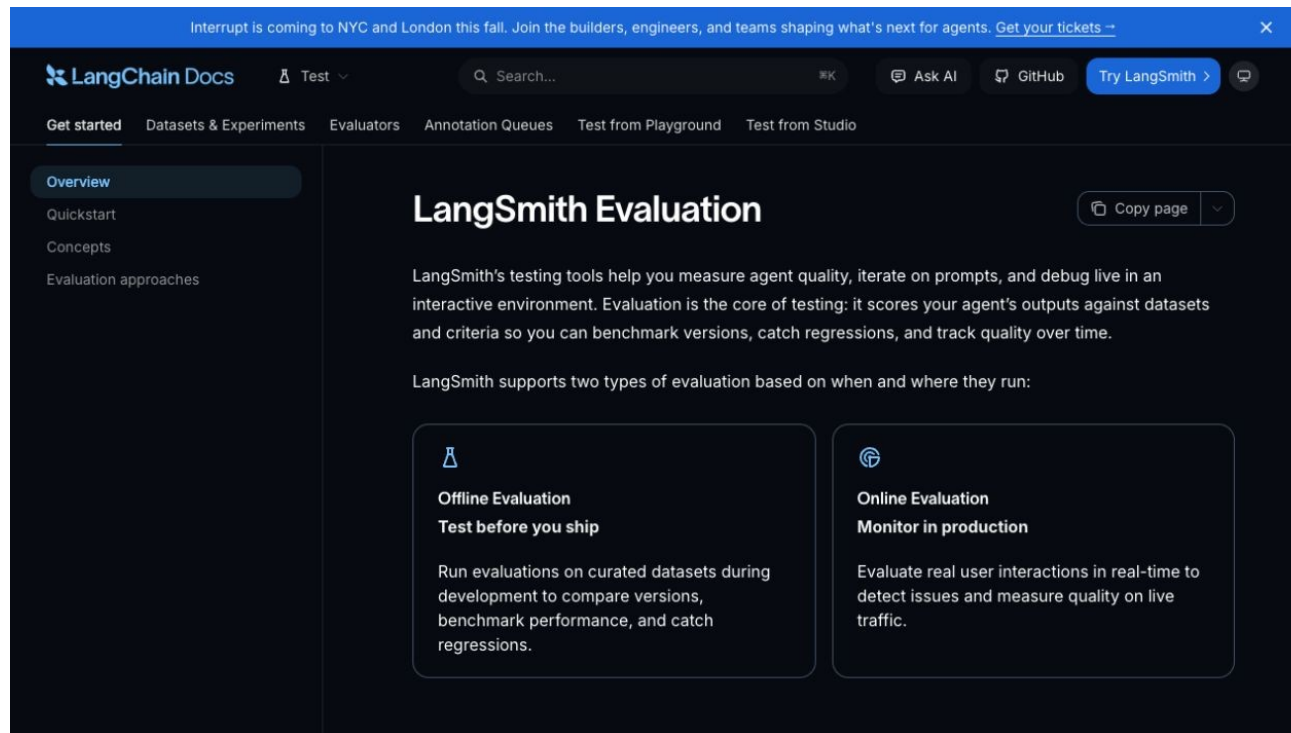
저장소 이해

→ 허용 범위 변경

→ focused test

→ diff 리뷰

→ 사람 merge



Codex에게는 목표보다 완료 조건을 더 구체적으로 줍니다

목표

새 평가 지표에 empty set과 threshold boundary test를 추가한다.

허용 경로

data/day5_eval/golden_review_findings.json
tests/test_course_services.py

완료 조건

fixture precision-recall-F1이 1.0이다.
precision 또는 recall 0.8 미만은 HOLD다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

생성된 patch는 diff와 test 증거를 보고 사람이 결정합니다

- 01 **Scope** — 허용 경로 밖 파일이 바뀌지 않았는가
- 02 **Focused test** — fixture precision·recall·F1 이 1.0이다.
- 03 **Boundary test** — precision 또는 recall 0.8 미만은 HOLD다.
- 04 **Human merge** — Codex 리뷰와 test 통과는 자동 승인이 아니다

강사가 먼저 정상 경로를 끝까지 실행합니다

- 01 — 열기: `src/course_services/eval_service.py`
- 02 — 구현 확인: `data/day5_eval/golden_review_findings.json`
- 03 — 실행: `python -m pytest -q tests/test_course_services.py -k offline_eval`
- 04 — 성공 신호: fixture precision·recall·F1이 1.0이다.

같은 저장소에서 이 명령을 실행합니다

```
$ python -m pytest -q tests/test_course_services.py -k offline_eval
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지

완료 여부는 화면이 아니라 결과 계약으로 확인합니다

STATUS

SUCCESS 또는 EXPECTED_FAILURE

ARTIFACT

experiment_result.json

사람이 확인할 세 줄

1. fixture precision·recall·F1이 1.0이다.
2. precision 또는 recall 0.8 미만은 HOLD다.
3. external_write=false

강사는 실패 한 건도 바로 재현하고 복구합니다

재현

LLM judge 한 개 점수를 객관적
정답처럼 사용한다.

복구

schema·line mapping·policy는 code
evaluator, 의미 품질은 human
sample로 나눈다.

확인: precision 또는 recall 0.8 미만은 HOLD다.

이제 같은 코드를 각자 실행하고 한 줄만 바꿉니다

열 파일

materials/day5/day5_service_lab.ipynb

수정 범위

notebook의 실습 변수 또는 fixture 한 줄 · src 전체를 복사해 바꾸지 않음

완료 기준

fixture precision·recall·F1이 1.0이다.
experiment_result.json 저장

STEP 1 · 입력과 설정을 확인합니다

파일: `src/course_services/eval_service.py`
변수: 실행 경로·provider·dry-run 여부

결과 파일: `experiment_result.json`

STEP 2 · 정상 경로를 실행합니다

```
python -m pytest -q tests/test_course_services.py -k offline_eval
```

성공 신호: fixture precision·recall·F1 이 1.0이다.

결과 파일: experiment_result.json

STEP 3 · 경계값을 바꿔 실패를 확인합니다

precision 또는 recall 0.8 미만은 HOLD다.

복구: schema-line mapping-policy는 code evaluator, 의미 품질은 human sample
로 나눈다.

결과에 error_code와 external_write=false가 보이면 완료

정상 case를 focused test로 확인합니다

NORMAL

fixture precision-recall-F1이 1.0이다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

가장 중요한 실패 조건을 test로 고정합니다

BOUNDARY

precision 또는 recall 0.8 미만은 HOLD다.

- 01 — raw traceback 대신 stable error_code가 남는가
- 02 — 외부 쓰기·자동 메일이 false인가
- 03 — 실패가 다음 차시에서 재현 가능한가

재직자는 작은 운영 문제 한 곳에 먼저 적용합니다

릴리스 후보가 기존 version보다 실제로 나아졌는지 비교

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — experiment_result.json을 운영 검토 자료로 사용

구직자는 제품 판단과 검증 증거를 함께 보여줍니다

릴리스 후보가 기존 version보다 실제로 나아졌는지 비교

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — experiment_result.json과 README 재현 절차를 제출

서비스로 운영하려면 이 네 가지 질문에 답해야 합니다

품질

무엇을 fixture precision·recall·F1 이 1.0이다.로 정의하는가

복구

schema·line mapping·policy는 code evaluator, 의미 품질은 human sample로 나눈다.

안전

어디에서 사람 승인과 external_write=false를 확인하는가

관측

experiment_result.json에서 어떤 status·error_code를 보는가

4차시를 마치기 전에 세 가지를 확인합니다

- 01 — 설명: 평가는 같은 dataset으로 baseline과 candidate를 비교해야 합니다이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_course_services.py -k offline_eval`
- 03 — 증거: `experiment_result.json`과 정상·실패 test 결과가 남아 있다.

다음 차시: 사람 피드백은 좋아요 버튼보다 판단 이유가 필요합니다

13:50-14:40 (쉬는 시간 17:10-17:40)

사람 피드백은 좋아요 버튼보다 판단 이유가 필요합니다

approve·edit·reject와 수정 이유를 label로 남긴다.

진행

강의 12분 · 강사 시연 10분 · 소프트웨어
실습 23분 · 실행 확인 5분

사용 파일

src/meeting_agent_workflow.py
src/langgraph_lab.py

확인할 결과

human_feedback.jsonl

5차시는 앞의 결과를 이어받아 다음 상태를 만듭니다

입력	——	experiment_result.json
이번 차시	——	사람 피드백은 좋아요 버튼보다 판단 이유가 필요합니다
결과	——	human_feedback.jsonl
다음 연결	——	운영 가능한 Agent는 개인정보와 장애 대응을 먼저 설명합니다

단순 thumbs-up은 어떤 오류를 고쳐야 하는지 알려주지 못한다.

approve·edit·reject와 수정 이유를 label로 남긴다.

이번 차시의 확인 결과: human_feedback.jsonl

이 차시가 끝나면 세 가지를 설명하고 실행할 수 있습니다

- 01 — approve·edit·reject와 수정 이유를 label로 남긴다.
- 02 — 사람 수정 내용을 곧바로 training data로 재사용한다. 왜 위험한지 설명한다.
- 03 — human_feedback.jsonl에서 정상과 실패 상태를 직접 확인한다.

처음 보는 용어는 이 네 가지 역할만 구분합니다

용어	이 수업에서 쓰는 뜻
Annotation	사람이 결과에 붙인 label
Feedback	실행 품질에 대한 평가
Queue	검토가 필요한 실행 목록
Reviewer agreement	검토자 판단 일치 정도

입력·처리·결과·검증을 분리하면 문제가 빨리 보입니다

INPUT

src/meeting_agent_workflow.py

PROCESS

HOLD sample → 근거 표시 → 사람 결정

OUTPUT

human_feedback.jsonl

VERIFY

approve·edit·reject가 reviewer·reason과 저장된다.
PII가 있는 raw content는 annotation export에서
제외된다.

실행 흐름은 다섯 단계로 고정합니다

01

HOLD sample

02

근거 표시

03

사람 결정

04

reason label

05

dataset 후보

마지막 판단은 human_feedback.jsonl에서 사람이 확인합니다.

코드보다 먼저 성공·실패 계약을 정합니다

- 정상 — approve·edit·reject가 reviewer·reason과 저장된다.
- 경계 — PI가 있는 raw content는 annotation export에서 제외된다.
- 외부 쓰기 — 기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출
- 기록 — status·error_code·입력 식별자·human_feedback.jsonl

어떤 파일을 열고 어디에서 결과를 확인하는지 먼저 봅시다

입력·fixture

```
src/meeting_agent_workflow.py
```

구현 코드

```
src/langgraph_lab.py
```

검증·test

```
web-demo/app.js
```

따라하기 notebook

```
materials/day5/day5_service_lab.ipynb
```

사람 피드백은 좋아요 버튼보다 판단 이유가 필요합니다

단순 thumbs-up은 어떤 오류를 고쳐야 하는지 알려주지 못한다.

동의·비식별·품질 검토 후 별도 dataset 승인을 거친다.

이 차시에서
버릴 오해

사람 수정 내용을
곧바로 training data로
재사용한다.

비슷해 보이는 선택도 책임과 검증 범위가 다릅니다

구분	역할	이번 차시의 판단 기준
Annotation	사람이 결과에 붙인 label	결정론 test로 먼저 확인
Feedback	실행 품질에 대한 평가	adapter 경계를 확인
Queue	검토가 필요한 실행 목록	비용·지연·권한을 확인
Reviewer agreement	검토자 판단 일치 정도	사람 판단과 운영 기록을 확인

가장 먼저 재현할 실패는 이것입니다

실패

사람 수정 내용을 곧바로 training data로 재사용한다.

사용자 영향

단순 thumbs-up은 어떤 오류를 고쳐야 하는지 알려주지 못한다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

복구는 재시도보다 먼저 중단 조건을 정합니다

- 01 — 탐지: PII가 있는 raw content는 annotation export에서 제외된다.
- 02 — 중단: 사람 수정 내용을 곧바로 training data로 재사용한다.
- 03 — 복구: 동의·비식별·품질 검토 후 별도 dataset 승인을 거친다.
- 04 — 증거: human_feedback.jsonl과 test 결과를 함께 남긴다.

Codex는 코드를 대신 쓰는 도구보다 검증 루프에 가깝습니다

feedback schema와 allowed label을 구현하고 unknown label을 차단한다.

저장소 이해

→ 허용 범위 변경

→ focused test

→ diff 리뷰

→ 사람 merge

WORKFLOW LAB

Pipeline Review Result

02

LANGGRAPH INTERRUPT

결과를 내보내기 전에 사람이 결정합니다.

REVIEW_REQUIRED

고객 문의 자동화 PoC 범위 회의

배송 지연과 반품 절차 안내만 1차 자동화 범위로 정하고, 외부 발행 전 사람이 검토한다.

ACTION ITEMS

공개 FAQ 30건 비식별 샘플 정리

서연 · 2026-08-27

근거 s12, s18

응답 JSON Schema와 실패 테스트 작성

준호 · 2026-08-28

근거 s27, s31

interrupt_payload.json

read only

```
{
  "question": "회의 결과를 승인·수정·거절하시겠습니까?",
  "request_id": "meeting-approve-001",
  "summary": "배송 지연과 반품 절차 안내만 1차 자동화 범위로 정하고, 외부 발행 전 사람이 검토한다.",
  "action_items": [
    {
      "task": "공개 FAQ 30건 비식별 샘플 정리",
      "owner": "서연",
      "due_date": "2026-08-27",
      "evidence_ids": [
        "s12",
        "s18"
      ]
    },
    {
      "task": "응답 JSON Schema와 실패 테스트 작성",
      "owner": "준호"
    }
  ]
}
```

Codex에게는 목표보다 완료 조건을 더 구체적으로 줍니다

목표

feedback schema와 allowed label을 구현하고 unknown label을 차단한다.

허용 경로

src/langgraph_lab.py
web-demo/app.js

완료 조건

approve·edit·reject가 reviewer·reason과 저장된다.
PII가 있는 raw content는 annotation export에서 제외된다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

생성된 patch는 diff와 test 증거를 보고 사람이 결정합니다

- 01 **Scope** — 허용 경로 밖 파일이 바뀌지 않았는가
- 02 **Focused test** — approve·edit·reject가 reviewer·reason과 저장된다.
- 03 **Boundary test** — PII가 있는 raw content는 annotation export에서 제외된다.
- 04 **Human merge** — Codex 리뷰와 test 통과는 자동 승인이 아니다

강사가 먼저 정상 경로를 끝까지 실행합니다

- 01 — 열기: `src/meeting_agent_workflow.py`
- 02 — 구현 확인: `src/langgraph_lab.py`
- 03 — 실행: `python -m src.langgraph_lab --decision all`
- 04 — 성공 신호: `approve·edit·reject`가 `reviewer·reason`과 저장된다.

같은 저장소에서 이 명령을 실행합니다

```
$ python -m src.langgraph_lab --decision all
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지

완료 여부는 화면이 아니라 결과 계약으로 확인합니다

STATUS

SUCCESS 또는 EXPECTED_FAILURE

ARTIFACT

human_feedback.jsonl

사람이 확인할 세 줄

1. approve·edit·reject가 reviewer·reason과 저장된다.
2. PII가 있는 raw content는 annotation export에서 제외된다.
3. external_write=false

강사는 실패 한 건도 바로 재현하고 복구합니다

재현

사람 수정 내용을 곧바로 training data
로 재사용한다.

복구

동의·비식별·품질 검토 후 별도
dataset 승인을 거친다.

확인: PII가 있는 raw content는 annotation export에서 제외된다.

이제 같은 코드를 각자 실행하고 한 줄만 바꿉니다

열 파일

materials/day5/day5_service_lab.ipynb

수정 범위

notebook의 실습 변수 또는 fixture 한 줄 · src 전체를 복사해 바꾸지 않음

완료 기준

approve·edit·reject가 reviewer·reason과 저장된다.
human_feedback.jsonl 저장

STEP 1 · 입력과 설정을 확인합니다

파일: `src/meeting_agent_workflow.py`
변수: 실행 경로·provider·dry-run 여부

결과 파일: `human_feedback.jsonl`

STEP 2 · 정상 경로를 실행합니다

```
python -m src.langgraph_lab --decision all
```

성공 신호: approve·edit·reject가 reviewer·reason과 저장된다.

결과 파일: human_feedback.jsonl

STEP 3 · 경계값을 바꿔 실패를 확인합니다

PI가 있는 raw content는 annotation export에서 제외된다.
복구: 동의·비식별·품질 검토 후 별도 dataset 승인을 거친다.

결과에 error_code와 external_write=false가 보이면 완료

정상 case를 focused test로 확인합니다

NORMAL

approve·edit·reject가 reviewer·reason과 저장된다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

가장 중요한 실패 조건을 test로 고정합니다

BOUNDARY

PI가 있는 raw content는 annotation export에서 제외된다.

- 01 — raw traceback 대신 stable error_code가 남는가
- 02 — 외부 쓰기·자동 메일이 false인가
- 03 — 실패가 다음 차시에서 재현 가능한가

재직자는 작은 운영 문제 한 곳에 먼저 적용합니다

운영자 검토 결과를 다음 prompt-rule 개선 후보로 연결

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — human_feedback.jsonl을 운영 검토 자료로 사용

구직자는 제품 판단과 검증 증거를 함께 보여줍니다

운영자 검토 결과를 다음 prompt-rule 개선 후보로 연결

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — human_feedback.jsonl과 README 재현 절차를 제출

서비스로 운영하려면 이 네 가지 질문에 답해야 합니다

품질

무엇을 approve·edit·reject가 reviewer·reason과 저장된다.로 정의하는가

복구

동의·비식별·품질 검토 후 별도 dataset 승인을 거친다.

안전

어디에서 사람 승인과 external_write=false를 확인하는가

관측

human_feedback.jsonl에서 어떤 status·error_code를 보는가

5차시를 마치기 전에 세 가지를 확인합니다

- 01 — 설명: 사람 피드백은 좋아요 버튼보다 판단 이유가 필요합니다이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m src.langgraph_lab --decision all`
- 03 — 증거: `human_feedback.jsonl`과 정상·실패 test 결과가 남아 있다.

다음 차시: 운영 가능한 Agent는 개인정보와 장애 대응을 먼저 설명합니다

14:40-15:30 (쉬는 시간 17:10-17:40)

운영 가능한 Agent는 개인정보와 장애 대응을 먼저 설명합니다

redaction·retention·access·incident checklist를 만든다.

진행

강의 12분 · 강사 시연 10분 · 소프트웨어
실습 23분 · 실행 확인 5분

사용 파일

src/observability_lab.py
AGENTS.md

확인할 결과

ops_checklist.md

6차시는 앞의 결과를 이어받아 다음 상태를 만듭니다

입력	————	human_feedback.jsonl
이번 차시	————	운영 가능한 Agent는 개인정보와 장애 대응을 먼저 설명합니다
결과	————	ops_checklist.md
다음 연결	————	배포 후보는 코드·문서·demo가 같은 버전을 가리켜야 합니다

회의·상담·코드에는 개인 정보와 회사 비밀이 동시에 들어갈 수 있다.

redaction·retention·access·incident checklist를 만든다.

이번 차시의 확인 결과: ops_checklist.md

이 차시가 끝나면 세 가지를 설명하고 실행할 수 있습니다

- 01 — redaction·retention·access·incident checklist를 만든다.
- 02 — 편의를 위해 원본 audio와 raw prompt를 무기한 외부에 보관한다. 왜 위험한지 설명한다.
- 03 — ops_checklist.md에서 정상과 실패 상태를 직접 확인한다.

처음 보는 용어는 이 네 가지 역할만 구분합니다

용어	이 수업에서 쓰는 뜻
PII	개인을 식별할 수 있는 정보
Redaction	민감 정보를 가리거나 제거
Retention	데이터 보관 기간
Incident	보안·품질 장애 사건

입력·처리·결과·검증을 분리하면 문제가 빨리 보입니다

INPUT

src/observability_lab.py

OUTPUT

ops_checklist.md

PROCESS

classify → minimize → redact

VERIFY

synthetic·deidentified만 외부 upload 후보가 된다.
local_only와 secret 탐지는 upload를 차단한다.

실행 흐름은 다섯 단계로 고정합니다

01

classify

02

minimize

03

redact

04

retain/delete

05

incident review

마지막 판단은 ops_checklist.md에서 사람이 확인합니다.

코드보다 먼저 성공·실패 계약을 정합니다

정상	——	<code>synthetic·deidentified</code> 만 외부 upload 후보가 된다.
경계	——	<code>local_only</code> 와 <code>secret</code> 탐지는 upload를 차단한다.
외부 쓰기	——	기본값 <code>false</code> · <code>dry-run</code> 과 사람 승인 뒤에만 별도 adapter 호출
기록	——	<code>status·error_code·입력 식별자·ops_checklist.md</code>

어떤 파일을 열고 어디에서 결과를 확인하는지 먼저 봅니다

입력·fixture

src/observability_lab.py

구현 코드

AGENTS.md

검증·test

.env.sample

따라하기 notebook

materials/day1/강사_회의음성_라이브데모_런북.md

운영 가능한 Agent는 개인정보와 장애 대응을 먼저 설명합니다

회의·상담·코드에는 개인 정보와 회사 비밀이 동시에 들어갈 수 있다.

목적·동의·최소수집·보존기한·삭제 경로를 배포 전 결정한다.

이 차시에서
버릴 오해

편의를 위해 원본 audio
와 raw prompt를
무기한 외부에 보관한
다.

비슷해 보이는 선택도 책임과 검증 범위가 다릅니다

구분	역할	이번 차시의 판단 기준
PII	개인을 식별할 수 있는 정보	결정론 test로 먼저 확인
Redaction	민감 정보를 가리거나 제거	adapter 경계를 확인
Retention	데이터 보관 기간	비용·지연·권한을 확인
Incident	보안·품질 장애 사건	사람 판단과 운영 기록을 확인

가장 먼저 재현할 실패는 이것입니다

실패

편의를 위해 원본 audio와 raw prompt를 무기한 외부에 보관한다.

사용자 영향

회의·상담·코드에는 개인 정보와 회사 비밀이 동시에 들어갈 수 있다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

복구는 재시도보다 먼저 중단 조건을 정합니다

- 01 — 탐지: local_only와 secret 탐지는 upload를 차단한다.
- 02 — 중단: 편의를 위해 원본 audio와 raw prompt를 무기한 외부에 보관한다.
- 03 — 복구: 목적·동의·최소수집·보존기한·삭제 경로를 배포 전 결정한다.
- 04 — 증거: ops_checklist.md과 test 결과를 함께 남긴다.

Codex는 코드를 대신 쓰는 도구보다 검증 루프에 가깝습니다

trace scrubber가 token·email·전화번호를 제거하는 정상·실패 test를 만든다.

저장소 이해
→ 허용 범위 변경
→ focused test
→ diff 리뷰
→ 사람 merge

The screenshot shows the Google Meet Help Center interface. At the top, there's a search bar and a '로그인' (Login) button. Below the navigation bar, the 'Gemini in Meet' section is highlighted. The main content area displays the article 'Google Meet의 자동 회의록' (Google Meet Automatic Meeting Summary). The article includes a '중요:' (Important) section with bullet points about requirements for using the feature, such as having a Google Workspace version or Google AI subscription, and a list of supported languages: English, French, German, Italian, Japanese, Korean, Portuguese, and Spanish. A note at the bottom states that the feature only supports one language at a time and that using multiple languages in a meeting is not supported. A right sidebar titled 'Help Center' lists various help topics, including 'Gemini 기반 Google Workspace 시작하기', 'Google Meet의 Gemini로 배경 이미지 만들기', 'Enhance your video & audio with Gemini in Google Meet', 'Google Meet의 자동 회의록', '오프라인 회의에 자동 회의록 사용하기', 'Meet에서 스마트 오디오 사용하기', 'Google Meet의 Gemini에게 물어보기', '통역 기능에 관해 알아보기', 'Calendar 및 Meet의 자동 회의록 설정', and 'Workspace의 Gemini 대화 기록 찾기 및 관리'.

Google Meet 고객센터

문제를 설명해 주세요.

로그인

Google Meet

Google Meet 시작하기 화상 회의 시작 또는 참여하기 **Gemini in Meet** 회의 중 작업 회의 종료 후 기능 사용 가능 여부 Google Meet 설정 Google Meet에서 통화하기

Google Meet의 자동 회의록

중요:

- 이 기능을 사용하려면 요건을 충족하는 Google Workspace 버전 또는 Google AI 요금제를 구독해야 합니다. [Gemini 기능 및 요금제 알아보기](#)
- 이 기능은 다음 언어로 사용할 수 있습니다.
 - 영어
 - 프랑스어
 - 독일어
 - 이탈리아어
 - 일본어
 - 한국어
 - 포르투갈어
 - 스페인어

도움말: 이 기능은 한 번에 하나의 언어만 지원합니다. 현재 한 회의에서 여러 언어를 사용하는 경우는 지원되지 않습니다. [자동 회의록의 언어를 변경하는 방법 알아보기](#)

Help Center

- Gemini 기반 Google Workspace 시작하기
- Google Meet의 Gemini로 배경 이미지 만들기
- Enhance your video & audio with Gemini in Google Meet
- Google Meet의 자동 회의록**
- 오프라인 회의에 자동 회의록 사용하기
- Meet에서 스마트 오디오 사용하기
- Google Meet의 Gemini에게 물어보기
- 통역 기능에 관해 알아보기
- Calendar 및 Meet의 자동 회의록 설정
- Workspace의 Gemini 대화 기록 찾기 및 관리

Codex에게는 목표보다 완료 조건을 더 구체적으로 줍니다

목표

trace scrubber가 token·email·전화번호를 제거하는 정상·실패 test를 만든다.

허용 경로

AGENTS.md
.env.sample

완료 조건

synthetic·deidentified만 외부 upload 후보가 된다.
local_only와 secret 탐지는 upload를 차단한다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

생성된 patch는 diff와 test 증거를 보고 사람이 결정합니다

- 01 **Scope** — 허용 경로 밖 파일이 바뀌지 않았는가
- 02 **Focused test** — synthetic·deidentified만 외부 upload 후보가 된다.
- 03 **Boundary test** — local_only와 secret 탐지는 upload를 차단한다.
- 04 **Human merge** — Codex 리뷰와 test 통과는 자동 승인이 아니다

강사가 먼저 정상 경로를 끝까지 실행합니다

- 01 — 열기: `src/observability_lab.py`
- 02 — 구현 확인: `AGENTS.md`
- 03 — 실행: `python -m pytest -q tests/test_observability_workflow.py`
- 04 — 성공 신호: `synthetic-deidentified`만 외부 upload 후보가 된다.

같은 저장소에서 이 명령을 실행합니다

```
$ python -m pytest -q tests/test_observability_workflow.py
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지

완료 여부는 화면이 아니라 결과 계약으로 확인합니다

STATUS

SUCCESS 또는 EXPECTED_FAILURE

ARTIFACT

ops_checklist.md

사람이 확인할 세 줄

1. synthetic-deidentified만 외부 upload 후보가 된다.
2. local_only와 secret 탐지는 upload를 차단한다.
3. external_write=false

강사는 실패 한 건도 바로 재현하고 복구합니다

재현

편의를 위해 원본 audio와 raw prompt
를 무기한 외부에 보관한다.

복구

목적·동의·최소수집·보존기한·삭제
경로를 배포 전 결정한다.

확인: local_only와 secret 탐지는 upload를 차단한다.

이제 같은 코드를 각자 실행하고 한 줄만 바꿉니다

열 파일

materials/day1/강사_회의음성_라이브데모_런북.md

수정 범위

notebook의 실습 변수 또는 fixture 한 줄 · src 전체를 복사해 바꾸지 않음

완료 기준

synthetic-deidentified만 외부 upload 후보가 된다.
ops_checklist.md 저장

STEP 1 · 입력과 설정을 확인합니다

파일: src/observability_lab.py
변수: 실행 경로·provider·dry-run 여부

결과 파일: ops_checklist.md

STEP 2 · 정상 경로를 실행합니다

```
python -m pytest -q tests/test_observability_workflow.py
```

성공 신호: synthetic·deidentified만 외부 upload 후보가 된다.

결과 파일: ops_checklist.md

STEP 3 · 경계값을 바꿔 실패를 확인합니다

local_only와 secret 탐지는 upload를 차단한다.
복구: 목적·동의·최소수집·보존기한·삭제 경로를 배포 전 결정한다.

결과에 error_code와 external_write=false가 보이면 완료

정상 case를 focused test로 확인합니다

NORMAL

synthetic-deidentified만 외부 upload 후보가 된다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

가장 중요한 실패 조건을 test로 고정합니다

BOUNDARY

local_only와 secret 탐지는 upload를 차단한다.

- 01 — raw traceback 대신 stable error_code가 남는가
- 02 — 외부 쓰기·자동 메일이 false인가
- 03 — 실패가 다음 차시에서 재현 가능한가

재직자는 작은 운영 문제 한 곳에 먼저 적용합니다

교육 데모에서 시작해 실제 고객 데이터로 넘어가기 위한 readiness gate

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — ops_checklist.md을 운영 검토 자료로 사용

구직자는 제품 판단과 검증 증거를 함께 보여줍니다

교육 데모에서 시작해 실제 고객 데이터로 넘어가기 위한 readiness gate

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — ops_checklist.md과 README 재현 절차를 제출

서비스로 운영하려면 이 네 가지 질문에 답해야 합니다

품질

무엇을 synthetic-deidentified만 외부 upload 후보가 된다.로 정의하는가

안전

어디에서 사람 승인과 external_write=false를 확인하는가

복구

목적·동의·최소수집·보존기한·삭제 경로를 배포 전 결정한다.

관측

ops_checklist.md에서 어떤 status_error_code를 보는가

6차시를 마치기 전에 세 가지를 확인합니다

- 01 — 설명: 운영 가능한 Agent는 개인정보와 장애 대응을 먼저 설명합니다이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_observability_workflow.py`
- 03 — 증거: ops_checklist.md과 정상·실패 test 결과가 남아 있다.

다음 차시: 배포 후보는 코드·문서·demo가 같은 버전을 가리켜야 합니다

배포 후보는 코드·문서·demo가 같은 버전을 가리켜야 합니다

test·README·sample data·Vercel demo를 release candidate로 묶는다.

진행

강의 12분 · 강사 시연 10분 · 소프트웨어
실습 23분 · 실행 확인 5분

사용 파일

README.md
web-demo/

확인할 결과

release_candidate

7차시는 앞의 결과를 이어받아 다음 상태를 만듭니다

입력	——	ops_checklist.md
이번 차시	——	배포 후보는 코드·문서·demo가 같은 버전을 가리켜야 합니다
결과	——	release_candidate
다음 연결	——	최종 발표는 모델 자랑보다 문제·증거·운영 결정을 보여줍니다

코드는 최신인데 화면과 문서가 이전 schema면 사용자는 첫 실행에서 막힌다.

test·README·sample data·Vercel demo를 release candidate
로 묶는다.

이번 차시의 확인 결과: release_candidate

이 차시가 끝나면 세 가지를 설명하고 실행할 수 있습니다

- 01 — test·README·sample data·Vercel demo를 release candidate로 묶는다.
- 02 — Vercel 화면만 동작하고 backend result contract와 다른 mock을 보여준다. 왜 위험한지 설명한다.
- 03 — release_candidate에서 정상과 실패 상태를 직접 확인한다.

처음 보는 용어는 이 네 가지 역할만 구분합니다

용어	이 수업에서 쓰는 뜻
Release candidate	배포 직전 검증 버전
Smoke test	핵심 경로의 빠른 점검
README	설치·실행·복구 안내
Demo	실제 입력과 결과를 보여주는 시연

입력·처리·결과·검증을 분리하면 문제가 빨리 보입니다

INPUT

README.md

PROCESS

focused tests → full suite → README

OUTPUT

release_candidate

VERIFY

local fixture로 end-to-end 결과와 demo JSON이
생성된다.
build 실패·test 실패·HOLD가 있으면 release candidate
가 아니다.

실행 흐름은 다섯 단계로 고정합니다

01

focused tests

02

full suite

03

README

04

demo build

05

human release

마지막 판단은 release_candidate에서 사람이 확인합니다.

코드보다 먼저 성공·실패 계약을 정합니다

정상	—— local fixture로 end-to-end 결과와 demo JSON이 생성된다.
경계	—— build 실패·test 실패·HOLD가 있으면 release candidate가 아니다.
외부 쓰기	—— 기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출
기록	—— status·error_code·입력 식별자·release_candidate

어떤 파일을 열고 어디에서 결과를 확인하는지 먼저 봅시다

입력·fixture

README.md

구현 코드

web-demo/

검증·test

scripts/run_day1_preflight.py

따라하기 notebook

materials/day5/day5_service_lab.ipynb

배포 후보는 코드·문서·demo가 같은 버전을 가리켜야 합니다

코드는 최신인데 화면과 문서가 이전 schema면
사용자는 첫 실행에서 막힌다.

Python output JSON을 demo의 단일 input으로 사용하고 schema version
을 표시한다.

이 차시에서
버릴 오해

Vercel 화면만 동작하고
backend result contract
와 다른 mock을
보여준다.

비슷해 보이는 선택도 책임과 검증 범위가 다릅니다

구분	역할	이번 차시의 판단 기준
Release candidate	배포 직전 검증 버전	결정론 test로 먼저 확인
Smoke test	핵심 경로의 빠른 점검	adapter 경계를 확인
README	설치·실행·복구 안내	비용·지연·권한을 확인
Demo	실제 입력과 결과를 보여주는 시연	사람 판단과 운영 기록을 확인

가장 먼저 재현할 실패는 이것입니다

실패

Vercel 화면만 동작하고 backend result contract와 다른 mock을 보여준다.

사용자 영향

코드는 최신인데 화면과 문서가 이전 schema면 사용자는 첫 실행에서 막힌다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

복구는 재시도보다 먼저 중단 조건을 정합니다

- 01 — 탐지: build 실패·test 실패·HOLD가 있으면 release candidate가 아니다.
- 02 — 중단: Vercel 화면만 동작하고 backend result contract와 다른 mock을 보여준다.
- 03 — 복구: Python output JSON을 demo의 단일 input으로 사용하고 schema version을 표시한다.
- 04 — 증거: release_candidate과 test 결과를 함께 남긴다.

Codex는 코드를 대신 쓰는 도구보다 검증 루프에 가깝습니다

기능 구현 후 README·test·demo
data가 같은 contract인지 diff로
검토한다.

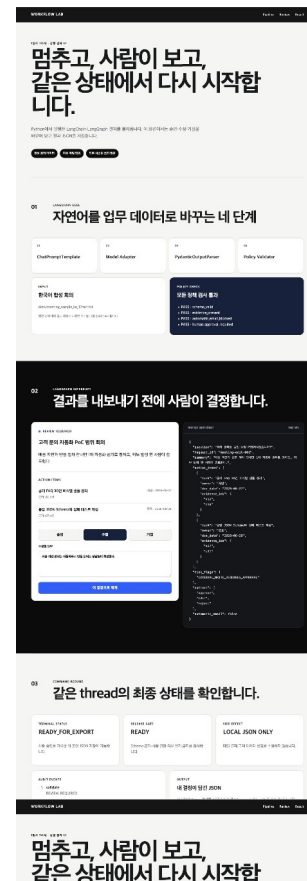
저장소 이해

→ 허용 범위 변경

→ focused test

→ diff 리뷰

→ 사람 merge



Codex에게는 목표보다 완료 조건을 더 구체적으로 줍니다

목표

기능 구현 후 README·test·demo data가 같은 contract인지 diff로 검토한다.

허용 경로

web-demo/
scripts/run_day1_preflight.py

완료 조건

local fixture로 end-to-end 결과와 demo JSON이 생성된다.
build 실패·test 실패·HOLD가 있으면 release candidate가 아니다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

생성된 patch는 diff와 test 증거를 보고 사람이 결정합니다

- 01 **Scope** — 허용 경로 밖 파일이 바뀌지 않았는가
- 02 **Focused test** — local fixture로 end-to-end 결과와 demo JSON이 생성된다.
- 03 **Boundary test** — build 실패·test 실패·HOLD가 있으면 release candidate가 아니다.
- 04 **Human merge** — Codex 리뷰와 test 통과는 자동 승인이 아니다

강사가 먼저 정상 경로를 끝까지 실행합니다

- 01 — 열기: README.md
- 02 — 구현 확인: web-demo/
- 03 — 실행: `python scripts/run_day1_preflight.py`
- 04 — 성공 신호: local fixture로 end-to-end 결과와 demo JSON이 생성된다.

같은 저장소에서 이 명령을 실행합니다

```
$ python scripts/run_day1_preflight.py
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지

완료 여부는 화면이 아니라 결과 계약으로 확인합니다

STATUS

SUCCESS 또는 EXPECTED_FAILURE

ARTIFACT

release_candidate

사람이 확인할 세 줄

1. local fixture로 end-to-end 결과와 demo JSON이 생성된다.

2. build 실패·test 실패·HOLD가 있으면 release candidate가 아니다.

3. external_write=false

강사는 실패 한 건도 바로 재현하고 복구합니다

재현

Vercel 화면만 동작하고 backend result contract와 다른 mock을 보여준다.

복구

Python output JSON을 demo의 단일 input으로 사용하고 schema version을 표시한다.

확인: build 실패·test 실패·HOLD가 있으면 release candidate가 아니다.

이제 같은 코드를 각자 실행하고 한 줄만 바꿉니다

열 파일

materials/day5/day5_service_lab.ipynb

수정 범위

notebook의 실습 변수 또는 fixture 한 줄 · src 전체를 복사해 바꾸지 않음

완료 기준

local fixture로 end-to-end 결과와 demo JSON이 생성된다.
release_candidate 저장

STEP 1 · 입력과 설정을 확인합니다

파일: README.md

변수: 실행 경로·provider·dry-run 여부

결과 파일: release_candidate

STEP 2 · 정상 경로를 실행합니다

```
python scripts/run_day1_preflight.py
```

성공 신호: local fixture로 end-to-end 결과와 demo JSON이 생성된다.

결과 파일: release_candidate

STEP 3 · 경계값을 바꿔 실패를 확인합니다

build 실패·test 실패·HOLD가 있으면 release candidate가 아니다.

복구: Python output JSON을 demo의 단일 input으로 사용하고 schema version
을 표시한다.

결과에 error_code와 external_write=false가 보이면 완료

정상 case를 focused test로 확인합니다

NORMAL

local fixture로 end-to-end 결과와 demo JSON이 생성된다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

가장 중요한 실패 조건을 test로 고정합니다

BOUNDARY

build 실패·test 실패·HOLD가 있으면 release candidate가 아니다.

- 01 — raw traceback 대신 stable error_code가 남는가
- 02 — 외부 쓰기·자동 메일이 false인가
- 03 — 실패가 다음 차시에서 재현 가능한가

재직자는 작은 운영 문제 한 곳에 먼저 적용합니다

내부 시연 URL과 재현 가능한 GitHub repo를 한 번에 전달

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — release_candidate을 운영 검토 자료로 사용

구직자는 제품 판단과 검증 증거를 함께 보여줍니다

내부 시연 URL과 재현 가능한 GitHub repo를 한 번에 전달

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — release_candidate과 README 재현 절차를 제출

서비스로 운영하려면 이 네 가지 질문에 답해야 합니다

품질

무엇을 local fixture로 end-to-end 결과와 demo JSON이 생성된다.로 정의하는가

안전

어디에서 사람 승인과 external_write=false를 확인하는가

복구

Python output JSON을 demo의 단일 input으로 사용하고 schema version을 표시한다.

관측

release_candidate에서 어떤 status_error_code를 보는가

7차시를 마치기 전에 세 가지를 확인합니다

- 01 — 설명: 배포 후보는 코드·문서·demo가 같은 버전을 가리켜야 합니다이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python scripts/run_day1_preflight.py`
- 03 — 증거: `release_candidate`과 정상·실패 test 결과가 남아 있다.

다음 차시: 최종 발표는 모델 자랑보다 문제·증거·운영 결정을 보여줍니다

16:20-17:10 (쉬는 시간 17:10-17:40)

최종 발표는 모델 자랑보다 문제·증거·운영 결정을 보여줍니다

3분 demo와 2분 질문으로 서비스 가치를 검증한다.

진행

강의 12분 · 강사 시연 10분 · 소프트웨어
실습 23분 · 실행 확인 5분

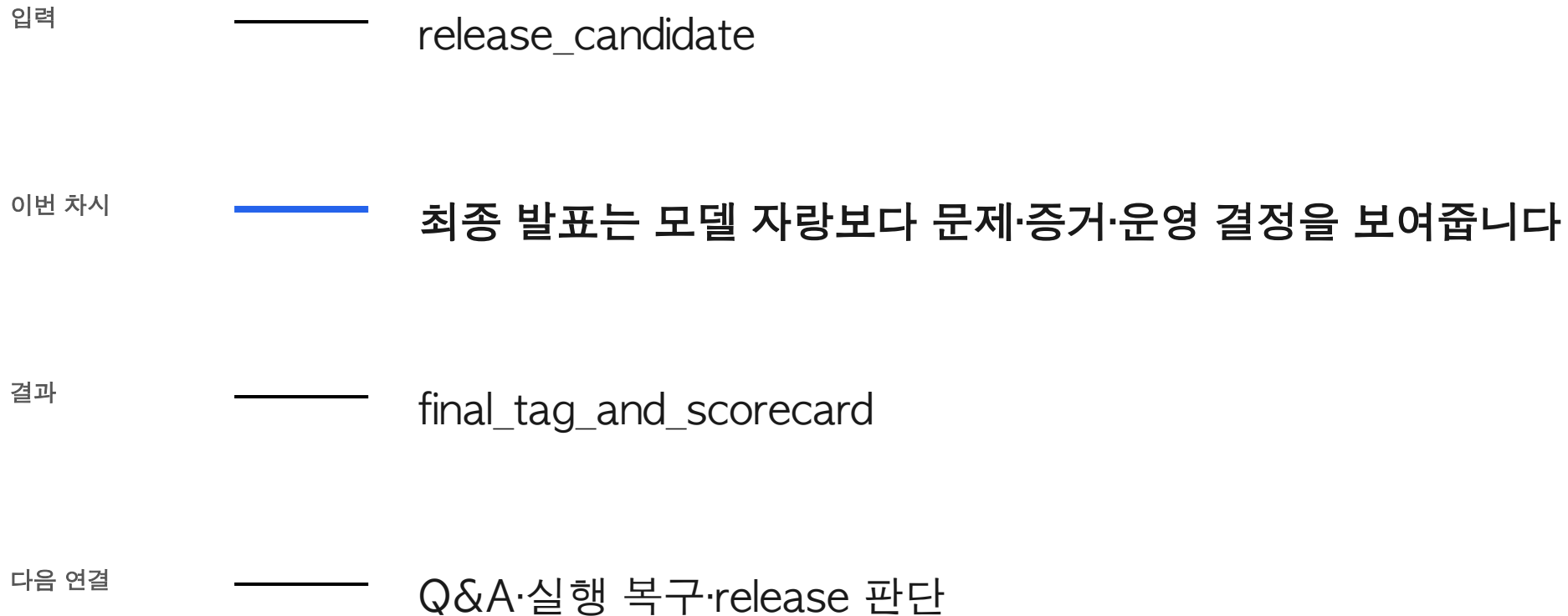
사용 파일

README.md
materials/day5/day5_service_lab.ipynb

확인할 결과

final_tag_and_scorecard

8차시는 앞의 결과를 이어받아 다음 상태를 만듭니다



기능 목록만 보여주면 왜 안전하고 운영 가능한지 판단할 수 없다.

3분 demo와 2분 질문으로 서비스 가치를 검증한다.

이번 차시의 확인 결과: final_tag_and_scorecard

이 차시가 끝나면 세 가지를 설명하고 실행할 수 있습니다

- 01 — 3분 demo와 2분 질문으로 서비스 가치를 검증한다.
- 02 — 성공 화면만 보여주고 실패·사람 승인·평가를 생략한다. 왜 위험한지 설명한다.
- 03 — final_tag_and_scorecard에서 정상과 실패 상태를 직접 확인한다.

처음 보는 용어는 이 네 가지 역할만 구분합니다

용어	이 수업에서 쓰는 뜻
Problem	누구의 어떤 반복 업무인지
Evidence	실행·test·평가 결과
Trade-off	선택으로 얻고 포기한 것
Next step	다음 검증 우선순위

입력·처리·결과·검증을 분리하면 문제가 빨리 보입니다

INPUT

README.md

OUTPUT

final_tag_and_scorecard

PROCESS

문제 30초 → live demo 90초 → 실패·HITL 30초

VERIFY

처음 보는 사람이 repo에서 demo를 재현할 수 있다.
비밀키·실제 고객 데이터·자동 외부 쓰기는 발표 자료에
없다.

실행 흐름은 다섯 단계로 고정합니다

01

문제 30초

02

live demo 90초

03

실패·HITL 30초

04

평가 30초

05

질문 2분

마지막 판단은 final_tag_and_scorecard에서 사람이 확인합니다.

코드보다 먼저 성공·실패 계약을 정합니다

정상	—— 처음 보는 사람이 repo에서 demo를 재현할 수 있다.
경계	—— 비밀키·실제 고객 데이터·자동 외부 쓰기는 발표 자료에 없다.
외부 쓰기	—— 기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출
기록	—— status·error_code·입력 식별자·final_tag_and_scorecard

어떤 파일을 열고 어디에서 결과를 확인하는지 먼저 봅시다

입력·fixture

README.md

구현 코드

materials/day5/day5_service_lab.ipynb

검증·test

output/day1-workflow/

따라하기 notebook

slides/

최종 발표는 모델 자랑보다 문제·증거·운영 결정을 보여줍니다

기능 목록만 보여주면 왜 안전하고 운영 가능한지 판단할 수 없다.

정상 한 건과 대표 HOLD 한 건을 나란히 보여준다.

이 차시에서
버릴 오해

성공 화면만 보여주고
실패·사람 승인·평가를
생략한다.

비슷해 보이는 선택도 책임과 검증 범위가 다릅니다

구분	역할	이번 차시의 판단 기준
Problem	누구의 어떤 반복 업무인지	결정론 test로 먼저 확인
Evidence	실행·test·평가 결과	adapter 경계를 확인
Trade-off	선택으로 얻고 포기한 것	비용·지연·권한을 확인
Next step	다음 검증 우선순위	사람 판단과 운영 기록을 확인

가장 먼저 재현할 실패는 이것입니다

실패

성공 화면만 보여주고 실패·사람 승인·평가를 생략한다.

사용자 영향

기능 목록만 보여주면 왜 안전하고 운영 가능한지 판단할 수 없다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

복구는 재시도보다 먼저 중단 조건을 정합니다

- 01 — 탐지: 비밀키·실제 고객 데이터·자동 외부 쓰기는 발표 자료에 없다.
- 02 — 중단: 성공 화면만 보여주고 실패·사람 승인·평가를 생략한다.
- 03 — 복구: 정상 한 건과 대표 HOLD 한 건을 나란히 보여준다.
- 04 — 증거: final_tag_and_scorecard과 test 결과를 함께 남긴다.

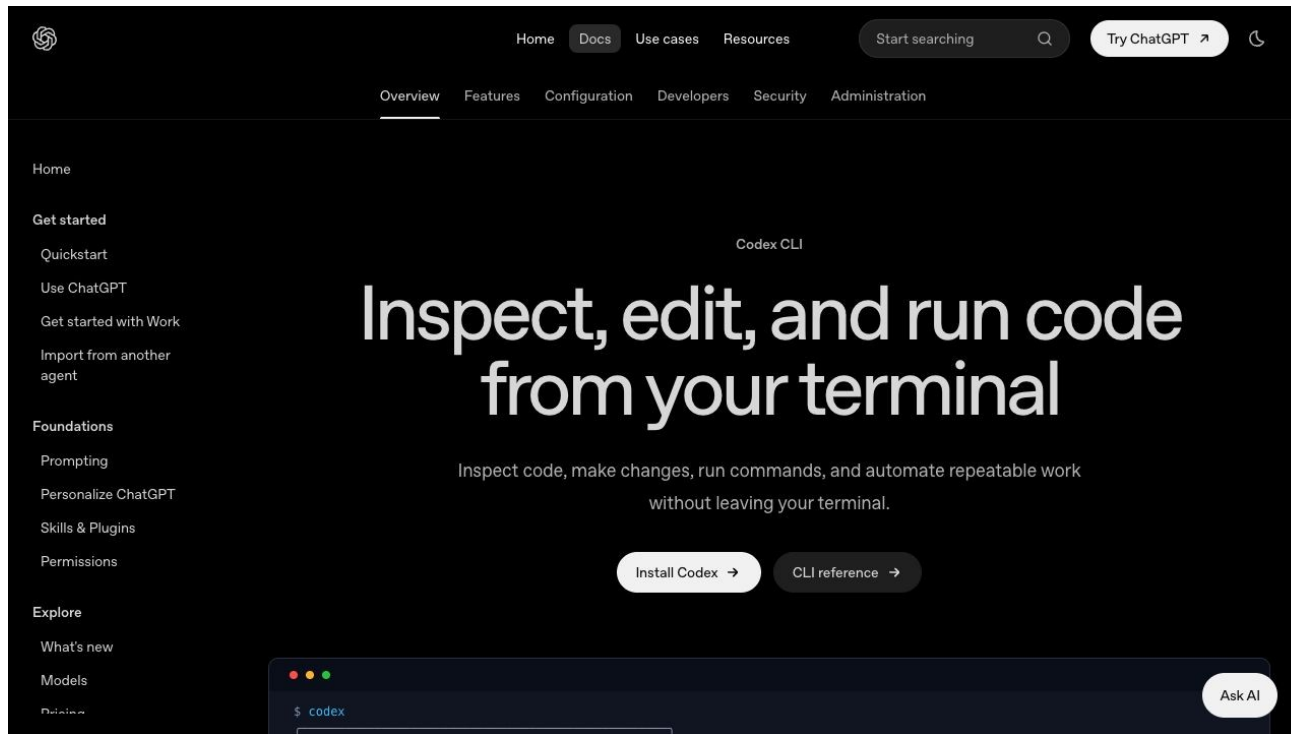
Codex는 코드를 대신 쓰는 도구보다 검증 루프에 가깝습니다

발표 전 저장소 구조

·README·test 증거를 다시
점검하고 누락만 보완한다.

저장소 이해

- 허용 범위 변경
- focused test
- diff 리뷰
- 사람 merge



Codex에게는 목표보다 완료 조건을 더 구체적으로 줍니다

목표

발표 전 저장소 구조·README·test 증거를 다시 점검하고 누락만 보완한다.

허용 경로

materials/day5/day5_service_lab.ipynb
output/day1-workflow/

완료 조건

처음 보는 사람이 repo에서 demo를 재현할 수 있다.
비밀키·실제 고객 데이터·자동 외부 쓰기는 발표 자료에 없다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

생성된 patch는 diff와 test 증거를 보고 사람이 결정합니다

- 01 **Scope** — 허용 경로 밖 파일이 바뀌지 않았는가
- 02 **Focused test** — 처음 보는 사람이 repo에서 demo를 재현할 수 있다.
- 03 **Boundary test** — 비밀키·실제 고객 데이터·자동 외부 쓰기는 발표 자료에 없다.
- 04 **Human merge** — Codex 리뷰와 test 통과는 자동 승인이 아니다

강사가 먼저 정상 경로를 끝까지 실행합니다

- 01 — 열기: README.md
- 02 — 구현 확인: materials/day5/day5_service_lab.ipynb
- 03 — 실행: `git status --short && python -m pytest -q`
- 04 — 성공 신호: 처음 보는 사람이 repo에서 demo를 재현할 수 있다.

같은 저장소에서 이 명령을 실행합니다

```
$ git status --short && python -m pytest -q
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지

완료 여부는 화면이 아니라 결과 계약으로 확인합니다

STATUS

SUCCESS 또는 EXPECTED_FAILURE

ARTIFACT

final_tag_and_scorecard

사람이 확인할 세 줄

1. 처음 보는 사람이 repo에서 demo를 재현할 수 있다.
2. 비밀키·실제 고객 데이터·자동 외부 쓰기는 발표 자료에 없다.
3. external_write=false

강사는 실패 한 건도 바로 재현하고 복구합니다

재현

성공 화면만 보여주고 실패·사람 승인·
평가를 생략한다.

복구

정상 한 건과 대표 HOLD 한 건을
나란히 보여준다.

확인: 비밀키·실제 고객 데이터·자동 외부 쓰기는 발표 자료에 없다.

이제 같은 코드를 각자 실행하고 한 줄만 바꿉니다

열 파일

slides/

수정 범위

notebook의 실습 변수 또는 fixture 한 줄 · src 전체를 복사해 바꾸지 않음

완료 기준

처음 보는 사람이 repo에서 demo를 재현할 수 있다.
final_tag_and_scorecard 저장

STEP 1 · 입력과 설정을 확인합니다

파일: README.md

변수: 실행 경로·provider·dry-run 여부

결과 파일: final_tag_and_scorecard

STEP 2 · 정상 경로를 실행합니다

```
git status --short && python -m pytest -q
```

성공 신호: 처음 보는 사람이 repo에서 demo를 재현할 수 있다.

결과 파일: final_tag_and_scorecard

STEP 3 · 경계값을 바꿔 실패를 확인합니다

비밀키·실제 고객 데이터·자동 외부 쓰기는 발표 자료에 없다.
복구: 정상 한 건과 대표 HOLD 한 건을 나란히 보여준다.

결과에 `error_code`와 `external_write=false`가 보이면 완료

정상 case를 focused test로 확인합니다

NORMAL

처음 보는 사람이 repo에서 demo를 재현할 수 있다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

가장 중요한 실패 조건을 test로 고정합니다

BOUNDARY

비밀키·실제 고객 데이터·자동 외부 쓰기는 발표 자료에 없다.

- 01 — raw traceback 대신 stable error_code가 남는가
- 02 — 외부 쓰기·자동 메일이 false인가
- 03 — 실패가 다음 차시에서 재현 가능한가

재직자는 작은 운영 문제 한 곳에 먼저 적용합니다

재직자는 업무 적용 계획, 구직자는 제품 판단과 검증 능력을 포트폴리오로
제시

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — final_tag_and_scorecard을 운영 검토 자료로 사용

구직자는 제품 판단과 검증 증거를 함께 보여줍니다

재직자는 업무 적용 계획, 구직자는 제품 판단과 검증 능력을 포트폴리오로 제시

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — final_tag_and_scorecard과 README 재현 절차를 제출

서비스로 운영하려면 이 네 가지 질문에 답해야 합니다

품질

무엇을 처음 보는 사람이 repo에서 demo를 재현할 수 있다.로 정의하는가

안전

어디에서 사람 승인과 external_write=false를 확인하는가

복구

정상 한 건과 대표 HOLD 한 건을 나란히 보여준다.

관측

final_tag_and_scorecard에서 어떤 status·error_code를 보는가

8차시를 마치기 전에 세 가지를 확인합니다

- 01 — 설명: 최종 발표는 모델 자랑보다 문제·증거·운영 결정을 보여줍니다이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `git status --short && python -m pytest -q`
- 03 — 증거: `final_tag_and_scorecard`과 정상·실패 test 결과가 남아 있다.

17:10-17:40 쉬는 시간 · 17:40-18:00 Q&A·실행 복구